

# Drizzle ORM Setup - Complete Summary

---

## ✓ Completed Tasks

---

### 1. Database Schema Created

Created 5 comprehensive schema files in `/db/schema/` :

#### **tasks.ts**

- Self-referencing parent-child relationship for task decomposition
- Status enum: pending, in\_progress, completed, failed, cancelled
- JSONB fields for metadata and results
- TypeScript types: `InsertTask` , `SelectTask`

#### **agents.ts**

- Agent type enum: planner, researcher, coder, reviewer, executor, custom
- JSONB fields for capabilities and config
- Indexes on type and name for efficient lookups
- TypeScript types: `InsertAgent` , `SelectAgent`

#### **executions.ts**

- Links tasks with agents via foreign keys
- Status enum: queued, running, completed, failed, cancelled, timeout
- Cascade delete on task removal
- Comprehensive indexes for query optimization
- JSONB fields for results, logs, and metrics
- TypeScript types: `InsertExecution` , `SelectExecution`

#### **llmConfigs.ts**

- Provider enum: openai, anthropic, google, azure, custom
- Priority-based configuration selection
- JSONB fields for routing rules and parameters
- Rate limiting support
- Multiple indexes for efficient querying
- TypeScript types: `InsertLlmConfig` , `SelectLlmConfig`

#### **context.ts**

- Links to executions with cascade delete
- Context type tracking (memory, tool\_output, intermediate\_result)
- Sequence number for ordered retrieval
- Indexes on execution\_id, type, and sequence
- TypeScript types: `InsertContext` , `SelectContext`

### 2. Database Configuration Updated

- ✓ Updated `db/db.ts` to include all new tables in schema
- ✓ Fixed `DATABASE_URL` encoding (replaced `#` with `%23` )

- ☒ Updated `db/schema/index.ts` to export all new schemas
- ☒ Existing `drizzle.config.ts` already properly configured

### 3. Package Scripts Added

Added to `package.json` :

```
{
  "db:push": "npx drizzle-kit push",
  "db:studio": "npx drizzle-kit studio"
}
```

Existing scripts:

- `db:generate` - Generate migrations from schema
- `db:migrate` - Apply migrations to database

### 4. Migration Generated

Generated migration file: `db/migrations/0000_odd_mongoose.sql`

Contains:

- 6 enum types
- 7 tables (5 new + 2 existing)
- 3 foreign key constraints with proper cascade behavior
- 13 indexes for query optimization
- Default values and NOT NULL constraints

### 5. Documentation Created

Created comprehensive documentation:

- `db/README.md` - Complete database schema documentation
- `scripts/migrate.ts` - Custom migration script for manual application
- This summary document

### 6. Git Commit

All changes committed with detailed commit message:

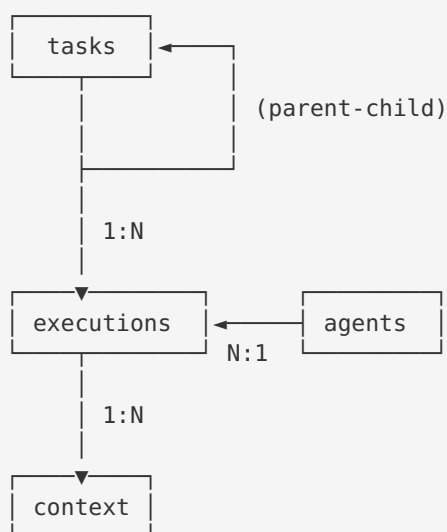
```
feat: Add Drizzle ORM schema for AI agent orchestration
```



## Schema Statistics

- **Tables Created:** 5 new tables
- **Total Tables:** 7 (including existing profiles tables)
- **Enum Types:** 6
- **Foreign Keys:** 3
- **Indexes:** 13
- **Relationships:** 5

## Database Relationships



### Important Notes

#### Database Connection Issue

During setup, encountered a network connectivity issue when trying to push the schema to Supabase. This is a temporary issue and doesn't affect the schema setup.

#### Alternative Migration Methods

Since `db:push` had connectivity issues, here are alternative ways to apply the migration:

##### Option 1: Supabase Dashboard (Recommended)

1. Go to your Supabase project dashboard
2. Navigate to **SQL Editor**
3. Copy contents of `db/migrations/0000_odd_mongoose.sql`
4. Paste and execute in the SQL Editor

##### Option 2: Using `psql`

```
PGPASSWORD='xcAE#uMjEEZ7FS1f' psql \
-h db.ktorvduzhojsvakixvcr.supabase.co \
-U postgres \
-d postgres \
-f db/migrations/0000_odd_mongoose.sql
```

##### Option 3: Custom Migration Script

```
npx tsx scripts/migrate.ts
```

##### Option 4: Retry `drizzle-kit` (when connectivity restored)

```
npm run db:push
```

## Next Steps

1. **Apply the migration** using one of the methods above
2. **Verify tables** in Supabase dashboard or Drizzle Studio ( `npm run db:studio` )
3. **Test database connection** by running queries
4. **Seed initial data** (optional - create agent configurations)
5. **Implement API routes** for CRUD operations

## Usage Examples

### Creating a Task

```
import { db } from "../db/db";
import { tasksTable } from "../db/schema";

const task = await db.insert(tasksTable).values({
  userId: "user_123",
  title: "Build a web scraper",
  description: "Create a Python web scraper",
  status: "pending"
}).returning();
```

### Querying with Relations

```
const executions = await db.query.executions.findMany({
  with: {
    task: true,
    agent: true,
    contexts: true
  },
  where: (executions, { eq }) => eq(executions.status, "running")
});
```

### Updating Execution Status

```
import { executionsTable } from "../db/schema";
import { eq } from "drizzle-orm";

await db.update(executionsTable)
  .set({
    status: "completed",
    endTime: new Date(),
    result: { output: "Task completed successfully" }
  })
  .where(eq(executionsTable.id, executionId));
```

## Files Changed

```

db/
├── schema/
│   ├── tasks.ts           (NEW)
│   ├── agents.ts         (NEW)
│   ├── executions.ts     (NEW)
│   ├── llmConfigs.ts     (NEW)
│   ├── context.ts        (NEW)
│   └── index.ts           (UPDATED)
├── migrations/
│   └── 0000_odd_mongoose.sql (NEW)
├── db.ts                  (UPDATED)
└── README.md              (NEW)

scripts/
└── migrate.ts             (NEW)

package.json              (UPDATED)
.env.local                 (UPDATED - DATABASE_URL encoding)
  
```

## Key Features

- **Type Safety:** Full TypeScript type inference for all tables
- **Relationships:** Properly defined one-to-many and self-referencing relations
- **Cascade Behavior:** Automatic cleanup on parent deletion
- **Indexes:** Optimized for common query patterns
- **Flexibility:** JSONB columns for dynamic data
- **Extensibility:** Easy to add new tables and relations

## Schema Highlights

### Task Decomposition

The `tasks` table supports hierarchical task breakdown with the self-referencing `parentTaskId` field, perfect for breaking down complex tasks into subtasks.

### Agent Orchestration

The `agents` table allows defining multiple agent types with different capabilities, enabling a flexible multi-agent system.

### Execution Tracking

The `executions` table provides comprehensive tracking of task execution with:

- Start and end times
- Results and error handling
- Retry counting
- Performance metrics
- Detailed logs

### LLM Routing

The `llmConfigs` table enables sophisticated LLM routing with:

- Multiple provider support

- Priority-based selection
- Custom routing rules
- Rate limiting

## Context Management

The `context` table maintains execution context and memory with:

- Ordered sequence tracking
- Type-based organization
- Automatic cleanup on execution deletion



## Additional Resources

---

- [Drizzle ORM Documentation](https://orm.drizzle.team/) (<https://orm.drizzle.team/>)
  - [Supabase PostgreSQL Guide](https://supabase.com/docs/guides/database) (<https://supabase.com/docs/guides/database>)
  - [Project PRD Documents](#) (./PRD-\*.md)
- 

**Setup Status:**  Complete (pending migration application to database)

**Last Updated:** November 14, 2025